

Beyond Static Summarization: Proactive Memory Extraction for LLM Agents

Chengyuan Yang[†] Zequn Sun^{†,*} Wei Wei[†] Wei Hu^{†,‡}

[†] State Key Laboratory for Novel Software Technology, Nanjing University, China

[‡] National Institute of Healthcare Data Science, Nanjing University, China
 {cyyang, weiw}.nju@gmail.com, {sunzq, whu}@nju.edu.cn

Abstract

Memory management is vital for LLM agents in long-term and personalized interactions. Most previous work studies how to retrieve and use memory, but pays less attention to how memory is extracted. We find two main limitations in existing methods. First, extraction is “ahead-of-time”: the agent saves information before it knows future tasks. A single summary prompt often mixes details, events, and relations, so useful information is lost. Second, extraction is usually one-off. Without verification, errors and hallucinations may stay in memory for a long time. To address these limitations, we propose ProMem, a proactive memory extraction framework. It separates details, events, and relations, and uses different extraction strategies for each type. It also checks completeness to recover missed events and verifies facts at the atomic level to reduce hallucinations. Experiments show that ProMem improves memory completeness and QA accuracy, while keeping a good balance between quality and token cost.

1 Introduction

Memory management is a cornerstone for large language model (LLM) based agents, enabling them to maintain long-term dialogue history and personalized information of users (Hu et al., 2025; Luo et al., 2024). To overcome the limitations of the LLM context window, existing research has relied on summary-based memory management (Chhikara et al., 2025). These methods compress historical interactions into concise summaries, aiming to preserve essential information while reducing computational overhead. As shown in Figure 1, most current summary-based methods focus heavily on the memory organization and utilization phase, often designing complex memory evolution loops and advanced memory retrieval mechanisms to handle stored memory summary (Kang et al.,

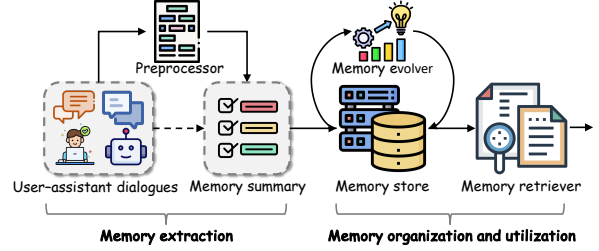


Figure 1: A general pipeline of summary-based memory management. It includes memory extraction, storage, update, and retrieval. Most previous work focuses on the later stages, while the extraction stage is simplified.

2025; Ke et al., 2025; Fang et al., 2026). However, the initial memory extraction is frequently overlooked. We argue that these methods suffer from two fundamental limitations:

First, memory extraction is “*ahead-of-time*”. The agent summarizes the dialogue history before knowing what the future task will be. This limitation often leads to a continuous accumulation of information loss. As the agent cannot predict future questions, it might easily discard small but crucial details or focus on irrelevant points. This makes the memory significantly less useful when the agent actually needs to invoke it later. Therefore, under this *ahead-of-time* constraint, it is important to explicitly define *what* the agent needs to remember. Furthermore, the extraction of distinct types of memory information cannot be treated as a single task handled by a unified extraction strategy.

Second, existing methods usually extract memory only once. We call this “*one-off extraction*”. If the agent makes a mistake or has a “hallucination” when it extracts memory for the first time, this error will stay in the memory and hurt the performance of agents in the future. We argue that feedback verification is essential for achieving high-quality memory management, yet current research has not paid enough attention to this vital step.

Figure 2 shows a motivating example. While

*Corresponding author

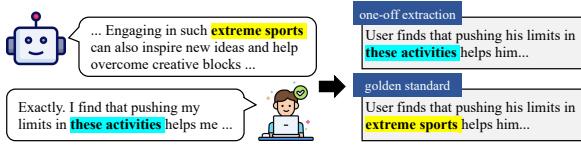


Figure 2: Example of info. loss in one-off extraction.

the user refers to “extreme sports” using the phrase “these activities,” a single-pass extractor fails to resolve this coreference, resulting in an ambiguous memory entry compared to the golden standard.

To resolve these limitations, we propose *proactive memory extraction*, called ProMem. It turns memory extraction into a multi-grained and iterative process. ProMem extracts different types of memory separately, then checks missing information and verifies suspicious facts against the original dialogue. This process helps recover omitted content and remove hallucinations. As a result, the final memory is more complete and more accurate. ProMem has four parts:

- **Multi-grained memory extraction:** The agent scans the unstructured dialogue history to separately extract fine-grained details, core events, and their relations to form a preliminary structured memory graph.
- **Completeness check via explicit matching:** The agent establishes a cross-reference matrix between the extracted events and initial memory drafts. If any valid event fails to align with the draft, the agent actively recovers and integrates this overlooked information to ensure memory completeness.
- **Fact verification via atomic decomposition:** The agent decomposes memory entries into finer atomic facts. It then utilizes natural language inference (NLI) to cross-check these facts against the source dialogue. It purifies invalid segments (hallucinations) without discarding the entire memory entry.
- **Relation verification and detail anchoring:** The agent generates probing questions to verify the logical relations (e.g., causal, temporal) between events. Finally, through semantic matching, the initially uncoupled fine details are precisely re-anchored to their corresponding verified events, yielding a high-fidelity, graph-structured final memory.

To evaluate the memory extraction capability of our ProMem, we use HaluMem (Chen et al., 2025) as our main dataset. We chose this dataset for two main reasons. First, our research focuses on solving the “missing information accumulation” problem caused by simple one-off extraction. HaluMem is specifically built to detect such phenomena. Second, the dataset contains complex user-agent interactions that require deep understanding, which allows us to evaluate whether our multi-grained extraction and atomic-level verification can construct more accurate, high-fidelity structured memories compared to traditional methods.

Our experiments show that ProMem outperforms baselines in both memory quality and QA accuracy. It is robust to token compression and remains effective when deployed with small language models.

2 Related Work

2.1 Summary-based Memory Management

Summary-based memory management is a common way to support long-horizon tasks under limited context windows (Zhu et al., 2025; Zhang et al., 2026; Hu et al., 2025; Liang et al., 2025). Its goal is to compress raw interaction history into short memory texts. Generative Agents (Park et al., 2023) uses hierarchical summarization. Memory-Bank (Zhong et al., 2024) adds a forgetting mechanism. MemGPT (Packer et al., 2024) handles the context limit with a hierarchical virtual memory system. Mem0 (Chhikara et al., 2025) manages user preferences and past interactions for personalized responses. In general, these methods keep summaries in working memory and store detailed logs in long-term storage. Many later studies focus on when to summarize and how to organize memory (Li et al., 2025; Kang et al., 2025; Yan et al., 2026; Fang et al., 2026). Although these methods reduce context load, they usually treat extraction as a passive and static step. Unlike them, ProMem focuses on making extraction more complete and reliable through proactive feedback.

2.2 LLM-based Information Extraction

Information extraction (IE) aims to transform unstructured text into structured formats. While traditional methods relied on supervised sequence labeling (Souza et al., 2019), the emergence of LLMs has shifted the paradigm towards generative IE.

Current research largely focuses on leveraging the instruction-following capabilities of LLMs

(Zhang et al., 2023; Wang et al., 2024b,a). UIE (Lu et al., 2022) provides a unified text-to-structure framework for different IE tasks. CodeKGC (Bi et al., 2024) and ChatIE (Wei et al., 2023) improve extraction with code generation or multi-turn interaction. However, most LLM-based IE methods focus on static text, such as encyclopedic or news articles. Our setting is different. ProMem extracts personalized user memory from multi-turn dialogue. This task needs not only fact extraction, but also consistency and completeness checking.

2.3 Self-Refinement and Iterative Prompting

Recent studies show that LLMs can improve their outputs through iterative feedback. Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) let the model review and revise its first output. Similar ideas are also used in reasoning and code generation to reduce one-pass errors (Don-Yehiya et al., 2023; Chen et al., 2023; Dhuliawala et al., 2024). ProMem is similar in spirit, but different in goal. Most self-refinement methods improve one task output, such as a poem or a code snippet. ProMem applies iteration to memory extraction in long-term dialogue. It focuses on two problems: missing information and hallucinations in memory.

2.4 Knowledge Graph Curation and Refinement

Knowledge graph curation studies how to extract and refine structured knowledge through completeness checking, grounding, fact validation, entity resolution, and knowledge fusion (Paulheim, 2017; Hogan et al., 2021; Ji et al., 2022; Dong et al., 2014). ProMem is related to this line of research but focuses on memory construction for long-term LLM agents. ProMem applies them to different memory types at different granularities. Details are extracted at the turn level, events are processed in topic-level contexts, and relations are verified across events.

3 Proactive Memory Extraction

In this section, we first introduce our motivation, and then present the proposed method ProMem.

3.1 Motivation

Current memory-augmented agents typically rely on a single-pass (one-off) extraction to compress lengthy dialogue history into concise memory. However, our empirical analysis (see Appendix A

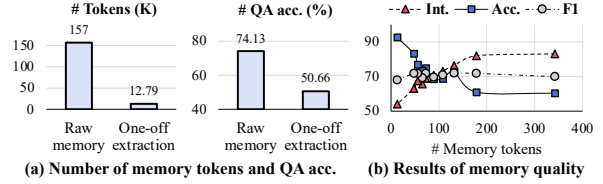


Figure 3: Limitations of one-off extraction. (a) It greatly reduces tokens, but QA accuracy drops by about 24%. (b) Increasing memory size brings only limited gains, which shows a structural bottleneck.

for setup details) on the naive RAG with raw memory and one-off extracted memory reveals two critical limitations of this pipeline:

- **Information loss and accuracy drop:** One-off extraction is efficient, but it loses important information. Figure 3(a) shows that it reduces memory from 157K to 12.79K tokens, but QA accuracy drops from 74.13% to 50.66%. This result shows that one scan cannot keep all necessary details.
- **Scaling ceiling of extracted memory quality:** A simple idea is extracting more facts. However, Figure 3(b) shows that a larger memory does not bring proportional F1 gains. The bottleneck is not only memory size. It also comes from the one-off extraction itself.

We propose a proactive memory extraction method called ProMem. It changes extraction from a static one-pass process to an active iterative process. It lets the agent re-check the dialogue, recover missed details, and remove hallucinations. This helps ProMem produce better memory with a reasonable quality-cost trade-off.

3.2 Multi-Grained Memory Disentanglement

The agent extracts memory at multiple granularities. Instead of using one prompt for everything, we divide the task into three sub-tasks. Given a dialogue history $D = \{t_1, t_2, \dots, t_n\}$, our goal is to build the basic parts of a structured memory graph.

Fine-grained detail extraction. The agent scans D and extracts atomic details, e.g., times, locations, and entities. We use prompt P_{detail} for this step:

$$F_{detail} = \text{LLM}(D, P_{detail}), \quad (1)$$

where F_{detail} represents the raw set of details.

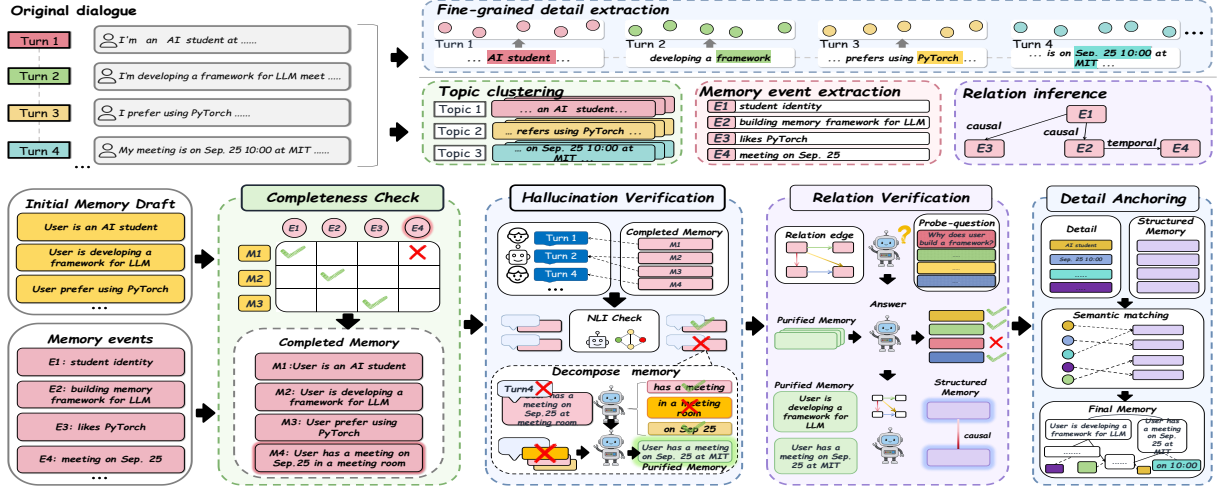


Figure 4: Overview of the ProMem framework. The pipeline consists of four main stages. (1) **Multi-grained Extraction**: The agent separately extracts fine details, macro events, and initial relations from the raw dialogue. (2) **Completeness Check**: It uses a cross-reference matrix to compare extracted events with the initial draft to recover missed information. (3) **Hallucination Verification**: It uses atomic decomposition and NLI to check facts and surgically remove errors. (4) **Relation Verification and Detail Anchoring**: It confirms logical edges and attaches fine details to the purified events. The final output is a high-quality, graph-structured memory.

Topic clustering and event abstraction. We then cluster the dialogue turns $t_i \in D$ into topic groups $C = \{c_1, c_2, \dots, c_m\}$. For each cluster c_j , the agent abstracts a set of higher-level events $E = \{e_1, e_2, \dots, e_k\}$. These events describe the user’s main states, actions, or stories.

Relation inference. Finally, to capture the dynamic progression of the user’s memory, we infer logical relations, such as causal or temporal links, between events in E . This gives a set of initial relation edges $R = \{(e_i, r, e_j)\}$.

Parallel execution. To reduce latency, detail extraction and event abstraction run in parallel. Relation inference is done after event abstraction.

3.3 Explicit Completeness Check

Many methods first produce an initial memory draft $M_{draft} = \{m_1, m_2, \dots, m_p\}$. However, this draft may miss important events. We therefore add an explicit completeness check to compare the abstracted events E with M_{draft} .

Cross-reference matrix. First, we map the abstracted events back to the draft. We encode both E and M_{draft} with a text embedding model. Let v_{e_i} and v_{m_j} be the embeddings of event e_i and draft entry m_j . We compute their cosine similarity:

$$S(e_i, m_j) = \frac{v_{e_i} \cdot v_{m_j}}{\|v_{e_i}\| \|v_{m_j}\|}. \quad (2)$$

For each event e_i , we find the most similar draft entry and compare the score with a threshold τ_{comp} :

$$\text{Match}(e_i) = \max_{m_j \in M_{draft}} S(e_i, m_j) > \tau_{comp}. \quad (3)$$

If $\text{Match}(e_i)$ is true, we consider that the event has been effectively captured in the draft.

Missing event recovery. If $\text{Match}(e_i)$ is false, then e_i is a missed event. We collect all missed events into a set E_{miss} . Then we merge them with the draft to form a more complete set M_{comp} :

$$M_{comp} = M_{draft} \cup E_{miss}. \quad (4)$$

3.4 Hallucination Verification via NLI

This module seeks to reduce hallucinations. Although M_{comp} has high recall, it may still contain false facts. We use a cascaded verification process based on natural language inference (NLI).

Turn-level grounding and NLI check. First, we match each memory entry $m \in M_{comp}$ to its most relevant dialogue turn $t_m \in D$. Then we use NLI to verify m against t_m . If the result is *Entailment*, we keep the entry. If the result is *Contradiction* or *Neutral*, we send it to the next stage.

Atomic decomposition. Discarding a whole entry may remove valid information. So we decompose the problematic entry m into atomic facts $A_m = \{a_1, a_2, \dots, a_q\} : A_m = \text{LLM}(m, P_{decomp})$, where P_{decomp} is the prompt for atomic decomposition.

NLI verification and LLM intervention. We then apply NLI to each atomic fact a_i against the anchored turn t_m . This helps us find the incorrect part of the memory:

- **Evidence verified:** If the NLI result is *Entailment*, we keep the atomic fact.
- **LLM correction or discard:** If the result is *Contradiction* or *Neutral*, we send a_i and t_m to the LLM. The LLM acts as an adjudicator: it either revises the fact or discards it.

Finally, we reconstruct the verified facts and merge them with the entries that passed the NLI check. This gives the purified memory $M_{purified}$.

3.5 Relation Verification and Detail Anchoring

This final module adds missing relations and attaches fine details back to events.

Necessity-driven relation verification. Flat memory lists often lose relations between events, e.g., causality or time order. But connecting every pair of events is expensive and often unnecessary. So we propose *necessity-driven* verification. For each candidate edge (m_u, r, m_v) between nodes $m_u, m_v \in M_{purified}$, the agent generates a probe question q_{rel} . We first test whether the agent can answer q_{rel} using only the flat entries in $M_{purified}$.

- **Implicit relation detected:** If the agent can answer the question, the relation is already clear, so we discard the edge.
- **Explicit edge supplementation:** If the agent cannot answer the question, we verify the relation against the original dialogue D . If the relation is supported, we add it to R_{valid} .

Detail anchoring. Finally, we attach the fine details F_{detail} back to the correct events. Recall that we extracted these details turn by turn in the first stage. Also, in the NLI verification stage, we already matched each purified event $m \in M_{purified}$ to a specific turn. Therefore, we can directly link them using the turn IDs. If a detail f and an event m come from the same turn, we anchor f to m . This step efficiently connects fine details to high-level events without complex embedding calculations. The final output is a graph-structured memory $G_{final} = (V, E_{edge}, A_{attr})$. Here, $V = M_{purified}$, $E_{edge} = R_{valid}$, and A_{attr} are the anchored details.

3.6 Discussion on Computational Overhead

We acknowledge that our method uses more tokens than one-pass summarization. This is mainly because it includes event recovery and verification. We believe this trade-off is reasonable for four reasons: First, memory errors are more costly than extra tokens. If the initial memory misses key facts or contains hallucinations, later tasks will also suffer. By spending more tokens at the start, we get cleaner and more reliable memory. Second, memory extraction is a “write-once, read-many” operation. We pay the extraction cost once, but the memory can be reused many times. So the average cost per use is acceptable. Third, memory extraction usually runs in the background. Unlike response generation, it does not directly block user interaction. So in this stage, quality is often more important than latency. Finally, the system can still be optimized. In practice, we do not need large models for every step. Small language models can handle decomposition, NLI checking, and semantic matching. We can also reduce token cost with token compression, as shown in Appendix C. We further report the component-level token cost, number of LLM calls, and runtime in Appendix B. The detailed results show that the additional overhead mainly comes from the write-time verification modules and is paid once during memory construction.

4 Experiments

4.1 Experimental Setup

Datasets. To comprehensively assess our framework, we select HaluMem (Chen et al., 2025) as our primary dataset. Our core contribution is to optimize the memory extraction process and mitigate hallucinations. HaluMem is specifically designed for this purpose. It provides fine-grained annotations to detect memory hallucinations and evaluate the consistency between extracted facts and the original dialogue. This aligns with our motivation. Besides, to demonstrate the robustness of our framework, we also conduct experiments on the widely used LongMemEval-S (Wu et al., 2025) and LoCoMo (Maharana et al., 2024).

Metrics. We report the following metrics:

- **Memory Integrity:** It measures the completeness of memory extraction. It calculates the recall of ground-truth facts, indicating how many user details are successfully extracted.

Method	Mem. Integrity	Mem. Accuracy	Mem. F1	QA Accuracy	# Tokens
Memobase (Memobase, Inc., 2025)	14.55	92.24	25.13	35.33	-
Supermemory (Supermemory, Inc., 2025)	41.53	90.32	56.89	54.07	-
MemOS (Li et al., 2025)	84.81	86.25	<u>79.70</u>	67.23	-
A-Mem (Xu et al., 2025)	-	-	-	43.02	36.71K
MemoryOS (Kang et al., 2025)	-	-	-	39.24	61.83K
Mem0 (Chhikara et al., 2025)	42.91	86.26	57.31	53.02	10.02K
LightMem ($r = 0.4$) (Fang et al., 2026)	55.21	<u>93.32</u>	69.37	56.60	2.40K
LightMem ($r = 0.6$) (Fang et al., 2026)	57.76	91.01	70.66	56.79	<u>6.58K</u>
LightMem ($r = 0.8$) (Fang et al., 2026)	58.09	92.70	71.42	56.45	<u>9.17K</u>
LightMem (Fang et al., 2026)	57.94	92.33	71.19	55.81	10.56K
ProMem	<u>81.37</u>	94.68	87.52	<u>63.43</u>	11.59K

Table 1: Performance (%) comparison of memory extraction and QA on the HaluMem benchmark. The best results are highlighted in **bold**. The second best results are underlined. “-” indicates that the metric is untestable, as the method does not support the corresponding evaluation.

- **Memory Accuracy:** This metric evaluates the correctness of the extracted entries. It penalizes hallucinations and factual errors, reflecting the reliability of the memory.
- **Memory F1:** It is a comprehensive metric of memory quality, calculated as the harmonic mean of Integrity and Accuracy.
- **QA Accuracy:** It measures whether the extracted memory can effectively support the agent in answering user queries.
- **# Tokens:** It refers to the average number of tokens consumed per session, i.e., the Total Cost of Ownership for the extraction process.

Baselines. We select three popular open-source projects to benchmark the practical performance of memory management systems: Memobase (Memobase, Inc., 2025), Supermemory (Supermemory, Inc., 2025) and MemOS (Li et al., 2025). We also compare our method with four leading methods in the research field: A-Mem (Xu et al., 2025), Mem0 (Chhikara et al., 2025), MemoryOS (Kang et al., 2025), and the SOTA method LightMem (Fang et al., 2026).

Implementation details. First, we use GPT-4o-mini to process the whole session. It quickly generates the initial memory draft. At the same time, we use a local Llama3.1-8B model for multi-grained memory disentanglement. The disentanglement runs in parallel with the initial extraction. Inside the disentanglement module, the fine-grained detail extraction also runs in parallel with the event extraction branch. For topic clustering, we follow the method in LightMem (Fang et al., 2026). After

clustering, Llama3.1-8B extracts macro events and their relations. For completeness check, we use the Qwen3-Embedding-8B model (Zhang et al., 2025). It calculates semantic similarities to find missing events. We set the completeness threshold τ_{comp} to 0.6. For hallucination verification, we use the roberta-large-mnli model to perform the NLI check. For relation verification, we use GPT-4o-mini to generate probe questions. During the QA phase, we use Qwen3-Embedding-8B to retrieve the top-20 memory entries and append timestamps to them. To save tokens, we use LLMingua-2 to compress the raw dialogue. Finally, we use GPT-4o combined with human review to evaluate the correctness of the generated answers. We run all experiments on a single NVIDIA A800 GPU.

4.2 Main Results

Table 1 presents the main results on HaluMem. First, our method achieves the best overall memory quality. It reaches the highest memory accuracy and F1. This proves that our framework effectively filters out hallucinations and extracts highly accurate facts. Second, ProMem strikes the best balance between memory integrity and accuracy. MemOS achieves the highest memory integrity and QA accuracy. However, its memory accuracy is relatively low. This means MemOS uses an aggressive strategy: it extracts many facts but introduces many errors. Memobase has good accuracy but very low integrity, meaning it misses too much information. In contrast, our method achieves the highest F1 score. We maintain the top accuracy while capturing very complete information (81.37% integrity).

Regarding the token cost, ProMem consumes 11.59K tokens. This cost is very close to the base-

Method	Mem. Int.	Mem. Acc.	QA Acc.	QA Halu.	QA Om.
ProMem (Full)	81.37	94.68	63.43	17.43	19.14
w/o CC	69.48	94.51	58.45	18.00	23.55
w/o HV	80.84	93.43	60.35	21.40	18.25
w/o DA	80.91	94.48	57.80	19.50	22.70
w/o Rel	81.15	94.42	61.12	18.40	20.48
One-off Extract	54.03	92.64	50.60	25.13	24.27

Table 2: Ablation study of key modules in ProMem.

line Mem0. In LightMem, r is the compression ratio. $r = 0.4$ means keeping 40% of the original tokens. Although LightMem($r = 0.4$) uses the fewest tokens (2.40K), its memory quality and downstream QA performance are much worse than ours. Even without compression, LightMem uses 10.56K tokens, but its results hardly improve. Its QA accuracy stays at 55.81%. At a similar token budget (11.59K vs. 10.56K), ProMem completely beats the full LightMem in all metrics. This proves that our high performance comes from our method, not from using more tokens. Thus, our method gets the best balance between quality and cost.

4.3 Ablation Study

Table 2 shows the ablation study results. “CC” means Completeness Check. “HV” means Hallucination Verification. “DA” means Detail Anchoring. “Rel” means Relational Edges. We also report QA Halu. and QA Om. First, the “*One-off Extract*” baseline performs worst. It has the lowest Integrity and QA Accuracy. Its QA Omission is also the highest. This shows that one-pass extraction misses much information. Second, the “*w/o CC*” variant shows the value of completeness check. Without CC, Integrity drops from 81.37% to 69.48%. QA Omission also increases. This shows that CC recovers missed events. Third, the “*w/o HV*” variant shows the value of fact checking. Without HV, Memory Accuracy is the lowest. QA Hallucination also rises to 21.40%. This shows that HV removes wrong facts. Fourth, the “*w/o DA*” shows the value of fine-grained details. Its memory scores stay close, but QA drops a lot. This means details are important for specific questions. Fifth, the “*w/o Rel*” variant shows the value of relation edges. Removing relations changes memory scores only a little, but QA drops to 61.12%. This shows that relations help reasoning. Finally, the full ProMem performs best. It gives the highest QA Accuracy. It also keeps hallucination and omission low. We also provide a component-level analysis in Appendix B. The results show that CC, HV, and REL detect different types of errors during memory

Method	Memory Int.	Memory Acc.	QA Acc.
Mem0	30.59	82.56	38.41
ProMem	43.09	82.32	49.15

Table 3: Results using SLMs.

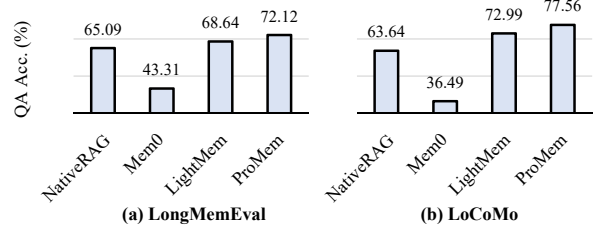


Figure 5: Results on LongMemEval-S and LoCoMo.

construction.

4.4 Results with SLMs

We test ProMem with Llama3.1-8B for extraction and QA. We want to know whether it still works with a smaller model. Table 3 shows the results. First, ProMem is much better than Mem0 on Memory Integrity. It can still recover missing details with a smaller model. Second, ProMem keeps similar Memory Accuracy. This means the extra facts do not add many hallucinations. Finally, QA Accuracy improves by 10.74%. These results show that ProMem still works well with SLMs.

4.5 Results on LongMemEval and LoCoMo

While HaluMem focuses on evaluating the integrity and faithfulness of extracted memory, we also need to verify the effectiveness of our method in supporting downstream applications. Therefore, we further conducted an additional experiment on LongMemEval-S (Wu et al., 2025) and LoCoMo (Maharana et al., 2024). We compare our ProMem with NativeRAG, Mem0, and the SOTA method LightMem. The results are presented in Figure 5.

First, ProMem achieves the best performance. It reaches a QA accuracy of 72.12 % on the dataset LongMemEval-S, surpassing all baselines. This proves that our proactive memory extraction framework effectively captures critical information needed for answering user questions.

Second, ProMem outperforms the SOTA method LightMem. It is worth noting that LightMem focuses on optimizing the retrieval mechanism (how to find memory), while our work focuses on optimizing the extraction quality (what to save). The fact that ProMem slightly beats LightMem suggests a fundamental insight: “*Better data is more impor-*

tant than better algorithms”. By ensuring the memory is complete and accurate, we can achieve superior results even without complex retrieval tricks.

Third, the failure of Mem0 highlights the importance of extraction quality. Mem0 performs significantly worse than NativeRAG. This indicates that Mem0’s aggressive summarization likely discarded too many details, making the memory less useful than the raw chunks used by NativeRAG. In contrast, ProMem significantly outperforms NativeRAG, demonstrating that *good memory management is beneficial, while bad memory management is worse than having no memory at all*. To provide a more fine-grained comparison, we report the category-level results on LongMemEval-S and LoCoMo in Appendix E.

4.6 ProMem as a Memory Foundation

We conduct an experiment to show that ProMem is a high-quality memory foundation. We use our extracted memory to replace the raw dialogue in existing baselines (NativeRAG, Mem0, and LightMem). We also replace the original memory of MemOS with ProMem memory while keeping its retrieval and management pipeline unchanged. Figure 6 shows the results.

First, ProMem substantially reduces the input size. As shown in Figure 6(d)–(f), the input tokens are reduced on all three datasets. For example, on LongMemEval-S, the input size drops from 101.619K tokens for the raw dialogue to 47.144K tokens with ProMem memory. On HaluMem, it drops from 159.91K to 37.45K tokens. This shows that ProMem can provide a much more compact memory representation.

Second, the QA accuracy remains high. Figure 6(a) and (b) show that the baselines still perform well when using our extracted memory. For example, LightMem achieves an accuracy of 70.12% on LongMemEval-S and 72.60% on LoCoMo. Moreover, Figure 6(c) shows that replacing the original MemOS memory with ProMem memory improves its QA accuracy on HaluMem from 67.23% to 68.46%, while keeping the retrieval and management pipeline unchanged. This controlled comparison further demonstrates the quality of the memory extracted by ProMem.

Overall, these results show that ProMem preserves useful information while substantially reducing the input length. It can also serve as a stronger memory input for an existing memory system. Therefore, ProMem provides an efficient and

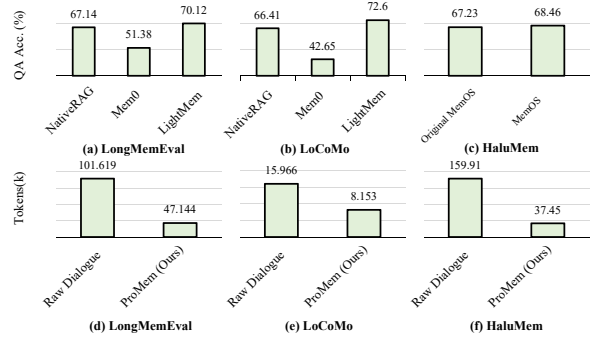


Figure 6: (a) and (b) show the QA accuracy when replacing the raw dialogue with our extracted memory on LongMemEval-S and LoCoMo. (c) compares the original MemOS with MemOS using ProMem memory on HaluMem. (d)–(f) show the reduction in input tokens on the three datasets. Overall, ProMem provides a compact and high-quality memory foundation for downstream memory systems.

high-quality memory foundation for downstream LLM agents.

4.7 Case Study

To intuitively show the advantage of our method, we present a case study in Table 4. The question asks how Sarah Garcia’s social network helped her career transition. Mem0 fails because its extraction is too coarse. It keeps only the general transition and misses the specific help from the network. So its answer is vague: “Positive influence”. This shows that traditional summarization often drops important details. In contrast, ProMem keeps fine-grained details. Its recurrent verification loop extracts key evidence such as “diverse perspectives” and “emotional support”. This more complete memory helps ProMem give a precise answer that matches the ground truth.

5 Conclusion and Future Work

In this paper, we identify the limitations of existing “one-off” memory summarization and propose proactive memory extraction. Our method introduces a recurrent feedback loop that enables agents to actively generate probe questions and verify facts against the dialogue. This helps the agent correct memory errors and supplement missing entries. Extensive experiments demonstrate that our ProMem achieves SOTA performance in both memory quality and QA tasks. ProMem can also be efficiently deployed using SLMs. In future work, we plan

	Question	Gold evidence	Gold answer	Extracted memory	Final answer
Mem0	What role did Sarah Garcia’s social network play in her career transition to Senior Manager at Oracle by Dec 10, 2025?	(1) Sarah Garcia’s social network expansion has been instrumental in providing insights and support during her transition to Senior Manager at Oracle. (2) Sarah Garcia expressed gratitude for the emotional support from friends Matthew and Linda, which has been crucial during her career transition.	Provided insights and emotional support.	Sarah Garcia is transitioning to a Senior Manager position at Oracle.	Positive influence on career growth..
ProMem	What role did Sarah Garcia’s social network play in her career transition to Senior Manager at Oracle by Dec 10, 2025?	(1) Sarah Garcia’s social network expansion has been instrumental in providing insights and support during her transition to Senior Manager at Oracle. (2) Sarah Garcia expressed gratitude for the emotional support from friends Matthew and Linda, which has been crucial during her career transition.	Provided insights and emotional support.	Event: Transition to Senior Manager/Support from friends Details: Friends Matthew and Linda/Dec 10, 2025/providing insights and support/at Oracle Relation: [Support from friends] → [Transition to Senior Manager].	Provided diverse perspectives and support.

Table 4: A case study comparing the memory extraction and QA performance of Mem0 and ProMem.

to extend our work to lifelong memory management. We will investigate memory updating and forgetting methods to manage memory.

Limitations

Although our ProMem method demonstrates superior performance in memory extraction and downstream QA tasks, we acknowledge several limitations that need to be addressed in future research. The first is computational overhead and latency. As discussed in Section 3.6, our method introduces a recurrent feedback loop involving self-questioning and supplementary extraction. Compared to traditional one-pass summarization, this iterative process inevitably increases token consumption and inference latency. While our analysis shows that the high quality of memory justifies this cost, this overhead might still be a bottleneck for strictly real-time or resource-constrained applications. The second is dependence on backbone LLM capabilities. The effectiveness of our “self-questioning” and “verification” modules relies heavily on the reasoning capabilities of the backbone LLM. If the model is too weak (e.g., unable to generate high-quality probing questions), the feedback loop may fail to correct errors. Although our experiments show that Llama3.1-8B performs reasonably well, exploring the lower bound of model size for this framework remains an open question.

Acknowledgments

This work was supported by National Natural Science Foundation of China (Nos. 62406136 and 62676187) and Natural Science Foundation of Jiangsu Province (No. BK20241246).

References

- Zhen Bi, Jing Chen, Yinuo Jiang, Feiyu Xiong, Wei Guo, Huajun Chen, and Ningyu Zhang. 2024. [Codekgc: Code language model for generative knowledge graph construction](#). *ACM Trans. Asian Low Resour. Lang. Inf. Process.*, 23(3):45.
- Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. 2025. [Halumem: Evaluating hallucinations in memory systems of agents](#). *CoRR*, abs/2511.03506.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. [Teaching large language models to self-debug](#). *ArXiv*, abs/2304.05128.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *CoRR*, abs/2504.19413.
- Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. [Chain-of-verification reduces hallucination in large language models](#). In *Findings of ACL*, pages 3563–3578.
- Shachar Don-Yehiya, Leshem Choshen, and Omri Abend. 2023. [Human learning by model feedback: The dynamics of iterative prompting with midjourney](#). In *EMNLP*, pages 4146–4161.
- Xin Dong, Evgeniy Gabrilovich, Jeremy Heitz, Wilko Horn, Ni Lao, Kevin Murphy, Thomas Strohmman, Shaohua Sun, and Wei Zhang. 2014. [Knowledge vault: a web-scale approach to probabilistic knowledge fusion](#). In *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’14, New York, NY, USA - August 24 - 27, 2014*, pages 601–610. ACM.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. [Lightmem: Lightweight and](#)

- efficient memory-augmented generation. In *International Conference on Learning Representations*, volume 2026, pages 98706–98729.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D’amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. [Knowledge graphs](#). *ACM Comput. Surv.*, 54(4).
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, and 28 others. 2025. [Memory in the age of ai agents](#). *CoRR*, abs/2512.13564.
- Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. 2022. [A survey on knowledge graphs: Representation, acquisition, and applications](#). *IEEE Trans. Neural Networks Learn. Syst.*, 33(2):494–514.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. [Memory OS of AI agent](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25961–25970, Suzhou, China. Association for Computational Linguistics.
- Cai Ke, Yiming Du, Bin Liang, Yifan Xiang, Lin Gui, Zhongyang Li, Baojun Wang, Yue Yu, Hui Wang, Kam-Fai Wong, and Ruifeng Xu. 2025. [Flexibly utilize memory for long-term conversation via a fragment-then-compose framework](#). In *EMNLP*.
- Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jiahao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, and 3 others. 2025. [Memos: An operating system for memory-augmented generation \(MAG\) in large language models](#). *CoRR*, abs/2505.22101.
- Jiafeng Liang, Hao Li, Chang Li, Jiaqi Zhou, Shixin Jiang, Zekun Wang, Changkai Ji, Zhihao Zhu, Runxuan Liu, Tao Ren, Jinlan Fu, See-Kiong Ng, Xia Liang, Ming Liu, and Bing Qin. 2025. [Ai meets brain: Memory systems from cognitive neuroscience to autonomous agents](#). *CoRR*, abs/2512.23343.
- Yaojie Lu, Qing Liu, Dai Dai, Xinyan Xiao, Hongyu Lin, Xianpei Han, Le Sun, and Hua Wu. 2022. [Unified structure generation for universal information extraction](#). In *ACL*, pages 5755–5772.
- Man Luo, Xin Xu, Zhuyun Dai, Panupong Pasupat, Mehran Kazemi, Chitta Baral, Vaiva Imbrasaitė, and Vincent Zhao. 2024. [Dr.ICL: Demonstration-retrieved in-context learning](#). *Data Intelligence*, 6(4):909–922.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). In *NeurIPS*.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. [Evaluating very long-term conversational memory of LLM agents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics.
- Memobase, Inc. 2025. Memobase: User profile-based long-term memory for ai chatbot applications. <https://github.com/memodb-io/memobase>. Accessed: 2025-01-04.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. [Memgpt: Towards llms as operating systems](#). *Preprint*, arXiv:2310.08560.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. [Llmllingua-2: Data distillation for efficient and faithful task-agnostic prompt compression](#). In *Findings of ACL*, pages 963–981.
- Joon Sung Park, Joseph C. O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. [Generative agents: Interactive simulacra of human behavior](#). In *UIST*, pages 2:1–2:22. ACM.
- Heiko Paulheim. 2017. [Knowledge graph refinement: A survey of approaches and evaluation methods](#). *Semantic Web*, 8(3):489–508.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: language agents with verbal reinforcement learning](#). In *NeurIPS*.
- Fábio Souza, Rodrigo Nogueira, and Roberto A. Lotufo. 2019. [Portuguese named entity recognition using BERT-CRF](#). *CoRR*, abs/1909.10649.
- Supermemory, Inc. 2025. Supermemory: Personalise your ai app with long-term memory api. <https://github.com/supermemoryai/supermemory>. Accessed: 2025-01-04.
- Jiaxin Wang, Lingling Zhang, Wee Sun Lee, Yujie Zhong, Liwei Kang, and Jun Liu. 2024a. [When phrases meet probabilities: Enabling open relation extraction with cooperating large language models](#). In *ACL*, pages 13130–13147.

Xinyi Wang, Wenzheng Zhao, Xiangrong Zhu, and Wei Hu. 2024b. [Can chatgpt solve relation extraction? an extensive assessment via design choice exploration](#). In *NLPCC*, volume 15360, pages 346–358.

Xiang Wei, Xingyu Cui, Ning Cheng, Xiaobin Wang, Xin Zhang, Shen Huang, Pengjun Xie, Jinan Xu, Yufeng Chen, Meishan Zhang, Yong Jiang, and Wenjuan Han. 2023. [Zero-shot information extraction via chatting with chatgpt](#). *CoRR*, abs/2302.10205.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. [LongMemEval: Benchmarking chat assistants on long-term interactive memory](#). In *ICLR*. OpenReview.net.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-mem: Agentic memory for llm agents](#). In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 17577–17604. Curran Associates, Inc.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schuetze, Volker Tresp, and Yunpu Ma. 2026. [Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12805–12825, San Diego, California, United States. Association for Computational Linguistics.

Ningyu Zhang, Jintian Zhang, Xiaohan Wang, Honghao Gui, Kangwei Liu, Yinuo Jiang, Xiang Chen, Shengyu Mao, Shuofei Qiao, Yuqi Zhu, Zhen Bi, Jing Chen, Xiaozhuan Liang, Yixin Ou, Runnan Fang, Zekun Xi, Xin Xu, Lei Li, Peng Wang, and 6 others. 2023. [Knowlm technical report](#).

Xiaohui Zhang, Zequn Sun, Chengyuan Yang, Yaqin Jin, Yazhong Zhang, and Wei Hu. 2026. [Actmem: Bridging the gap between memory retrieval and reasoning in llm agents](#). *Preprint*, arXiv:2603.00026.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. [Qwen3 embedding: Advancing text embedding and reranking through foundation models](#). *CoRR*, abs/2506.05176.

Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. [Memorybank: Enhancing large language models with long-term memory](#). In *AAAI*, pages 19724–19731.

Rongzhi Zhu, Xiangyu Liu, Zequn Sun, Yiwei Wang, and Wei Hu. 2025. [Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering](#). In *ACL*, pages 22362–22375.

Model	Hallucination by HV	Missing Events by CC	Missing Relations by REL
GPT-4o-mini	108 / 1,284 (8.41%)	80 / 682 (11.73%)	12 / 472 (2.54%)
Llama3.1-8B	348 / 2,199 (15.83%)	114 / 682 (16.71%)	17 / 472 (3.60%)

Table 5: Component-level errors detected by different verification modules. HV, CC, and REL denote Hallucination Verification, Completeness Check, and Relation Verification, respectively.

A Setup for Empirical Study

To investigate the performance bottlenecks of traditional methods, we conduct a series of empirical experiments on the HaluMem dataset (Chen et al., 2025). While conventional memory management typically performs extraction at the session level (where all dialogue turns are combined into a single input), we systematically vary the extraction granularity to observe its impact. Specifically, we evaluate extraction intervals ranging from every 2 turns to 4, 6, and 8 turns, eventually reaching the entire session (the standard baseline). To isolate the impact of granularity, the model strictly operates in a one-off extraction mode across all settings, without any feedback or iterative verification.

Regarding evaluation, we adopt the following metrics as in the main experiments: Memory Integrity (Int.) represents completeness, while Memory Accuracy (Acc.) reflects the factual correctness of the extracted content. We also report the F1 score, the harmonic mean of Integrity and Accuracy, to provide a comprehensive assessment of the overall memory quality.

B Component-level Analysis

We further analyze ProMem from two aspects: the problems detected by different verification modules and their token costs.

Detected errors. Table 5 reports the errors detected by Hallucination Verification (HV), Completeness Check (CC), and Relation Verification (REL).

The results show that the extraction process contains different types of errors. Both models produce unsupported facts, missing events, and missing relations. This motivates the use of dedicated verification modules for different error types.

Component-level cost. Table 6 reports the average token cost of each component.

Among the verification modules, DA has the largest token cost because it links fine-grained de-

Model	Initial Draft	CC	HV	DA	REL	Total
GPT-4o-mini	3.118K (26.90%)	1.747K (15.08%)	1.503K (12.97%)	3.055K (26.36%)	2.165K (18.68%)	11.59K
Llama3.1-8B	3.691K (27.40%)	1.822K (13.53%)	2.050K (15.22%)	3.330K (24.72%)	2.570K (19.08%)	13.47K

Table 6: Average token cost per session for different components of ProMem. CC, HV, DA, and REL denote Completeness Check, Hallucination Verification, Detail Anchoring, and Relation Verification, respectively.

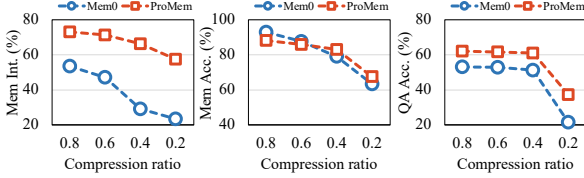


Figure 7: Performance w.r.t. compression ratios.

tails back to the corresponding events. CC and REL also introduce additional cost for recovering missing events and verifying relations. For the GPT-4o-mini setting, ProMem requires 4.26 LLM calls and 21.38 seconds per session on average. These costs are incurred at write time, while the resulting memory can be reused for later queries.

C Results with Token Compression

As discussed in Section 3.6, our extraction method inevitably introduces higher token overhead compared to one-pass methods. To address this efficiency concern and explore the practical deployability of ProMem, we conducted a token compression experiment. We discarded unimportant tokens from the dialogue history at different compression ratios (0.8, 0.6, 0.4, 0.2) and compared the performance of ProMem against the baseline Mem0. We follow LightMem and use LLMLingua-2 (Pan et al., 2024) as the compression model. The results are shown in Figure 7.

First, our ProMem demonstrates exceptional robustness to information loss. As the compression ratio drops from 0.8 to 0.2 (meaning 80% of the dialogue tokens are discarded), our method maintains a remarkably stable performance. Specifically, at a low compression ratio of 0.2, ProMem still achieves a QA score of 37.20% and memory integrity of 57.20%. This indicates that our self-questioning and supplementary extraction can successfully extract high-quality memory even from highly compressed inputs.

Second, ProMem significantly outperforms the baseline in low-resource settings. In contrast, Mem0 is highly sensitive to token reduction. As shown in the Figure 7, when the ratio drops to 0.2,

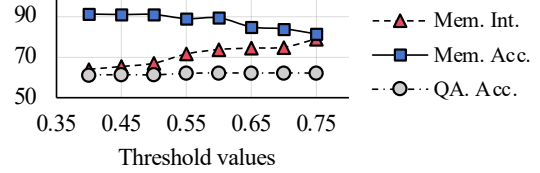


Figure 8: Performance w.r.t. different values of τ_{comp} .

the performance of Mem0 collapses. Its memory integrity drops to 23.28% and QA to 21.34%.

These results strongly imply that *we can deploy ProMem with a high compression rate in real-world applications*. By aggressively compressing the input tokens, we can reduce the token consumption by 60% while still achieving superior performance compared to full-input baselines. This effectively resolves the concern regarding computational costs.

D Sensitivity Analysis on τ_{comp}

To further evaluate the robustness of ProMem, we conduct a sensitivity analysis on the threshold τ_{comp} . Figure 8 shows the variations in Memory Integrity, Memory Accuracy, and downstream QA Accuracy. We observe that increasing τ_{comp} effectively enhances the completeness (Integrity) of the extracted memory by promoting supplementary extractions. Although this leads to a slight decrease in precision (Accuracy), the downstream QA performance stays consistent, fluctuating only marginally. This stability demonstrates that the proactive verification loop in ProMem can effectively manage the information flow even with different threshold settings. The results justify our choice of $\tau_{comp} = 0.6$, which serves as a sweet spot for maintaining high-quality memory without excessive token overhead.

E Category-level Results on LongMemEval-S and LoCoMo

Tables 7 and 8 report the category-level QA results. On LongMemEval-S, ProMem outperforms LightMem in five of the six categories and achieves similar performance on Knowledge-Update. On LoCoMo, ProMem achieves the best performance in all four categories. These results show that the

Method	Temporal	Multi-Session	Knowledge-Update	Single-User	Single-Assistant	Single-Preference	Overall
Naive RAG	39.85 $\pm$ 0.28	48.48 $\pm$ 0.32	67.95 $\pm$ 0.90	90.00 $\pm$ 0.39	98.21 $\pm$ 1.11	53.33 $\pm$ 0.21	65.09 $\pm$ 0.49
Mem0	29.95 $\pm$ 0.42	36.01 $\pm$ 0.41	59.92 $\pm$ 0.32	71.23 $\pm$ 0.45	30.86 $\pm$ 0.57	49.81 $\pm$ 0.41	43.31 $\pm$ 0.42
LightMem	67.18 $\pm$ 0.10	71.74 $\pm$ 0.25	83.12 $\pm$ 0.47	87.14 $\pm$ 0.37	32.14 $\pm$ 0.32	68.18 $\pm$ 0.11	68.64 $\pm$ 0.26
ProMem	69.43 $\pm$ 0.20	73.51 $\pm$ 0.22	83.10 $\pm$ 2.11	92.85 $\pm$ 0.24	42.67 $\pm$ 0.08	76.51 $\pm$ 0.14	72.12 $\pm$ 0.49

Table 7: Category-level QA accuracy (%) on LongMemEval-S. The best result in each column is shown in bold, and the second-best result is underlined.

Method	Single-Hop	Temporal	Open-Domain	Multi-Hop	Overall
Naive RAG	70.99 $\pm$ 0.43	56.39 $\pm$ 0.70	47.92 $\pm$ 0.70	55.32 $\pm$ 0.73	63.64 $\pm$ 0.56
Mem0	38.41 $\pm$ 0.26	37.07 $\pm$ 0.35	34.38 $\pm$ 0.73	30.85 $\pm$ 0.67	36.49 $\pm$ 0.38
LightMem	76.68 $\pm$ 0.49	70.58 $\pm$ 0.53	66.70 $\pm$ 0.65	66.87 $\pm$ 0.28	72.99 $\pm$ 0.47
ProMem	80.50 $\pm$ 0.20	74.18 $\pm$ 0.12	69.51 $\pm$ 0.28	75.38 $\pm$ 0.10	77.56 $\pm$ 0.17

Table 8: Category-level QA accuracy (%) on LoCoMo. The best result in each column is shown in bold, and the second-best result is underlined.

improvement of ProMem is consistent across different question types.

F Prompt Details

To facilitate reproducibility, we present the prompt templates used in our ProMem framework.

Stage 1: Multi-Grained Disentanglement. The prompts for fine-grained detail extraction, event abstraction, and relation inference are shown in Figures 9–11.

Stage 2: Completeness Check. The prompt for initial memory draft extraction is shown in Figure 12.

Stage 3: Hallucination Verification. The prompts for atomic decomposition and LLM intervention are shown in Figures 13 and 14.

Stage 4: Relation Verification. The prompt for relation probing is shown in Figure 15.

Note that the placeholders enclosed in curly braces (e.g., “{user_name}” and “{memory_entry}”) are replaced by actual data during inference.

You are a micro-detail extractor.

Dialogue Turn: {current_turn}

Your task: Extract all fine-grained, atomic details from this specific dialogue turn. Focus heavily on specific entities, times, locations, and explicit object names.

Rules:

- Do NOT summarize the overarching event. Only extract the specific attribute fragments.
- Examples of good details: “10:00 AM”, “at MIT”, “prefers PyTorch”, “AI student”.
- Output as a simple comma-separated list of short phrases.

Output format: [“detail 1”, “detail 2”, ...]

Figure 9: Prompt for Fine-grained Detail Extraction (P_{detail}).

You are an overarching event summarizer.

We have grouped several dialogue turns that share the same topic.

Topic-clustered Dialogue: {clustered_turns}

Your task: Abstract the core “Memory Events” from this clustered dialogue. A memory event should represent a macro-level action, status, or narrative of the user (“{user_name}”).

Rules:

- Ignore fragmented times or locations unless they define the event. Focus on the “Big Picture”.
- Write each event as a clear, standalone sentence in the third person.

Output format: Just a list of macro events, one per line.

Figure 10: Prompt for Topic-based Event Abstraction.

You are a structural logic analyzer.
Below is a list of macro memory events extracted from a user's dialogue history.
Extracted Events: {event_list}
Your task: Analyze the logical relationships between these events. Identify any explicit or highly probable "Causal" (A leads to B) or "Temporal" (A happens before B) connections.
Rules:
- Only infer relationships that are strongly supported by common sense or the event context.
- If two events are unrelated, do not force a connection.
Output format: Output strictly as a JSON list of arrays: [["Event_A_ID", "relation_type", "Event_B_ID"], ...]

Figure 11: Prompt for Relation Edge Inference.

You are a Personal Fact Extraction Engine.
Your task is to analyze conversation logs to identify and catalog every piece of factual information regarding the user ("{user_name}"). The goal is to convert dialogue fragments into explicit, self-contained statements to form an initial memory draft.
Important instructions:
1. Process the user messages and decide whether they contain factual information.
- Extract ANY assertion of fact, action, state, or preference.
- If no useful information is provided (e.g., pure greeting or filler), skip it.
2. Perform light contextual completion so that each fact is a clear standalone statement.
Example:
"User: Went to the gym." → "User went to gym."
3. **CRITICAL RULE:** Always use the name "{user_name}" as the subject instead of "User", "He", "She", or "I".
Reminder: Be exhaustive. Output the extracted facts as a clear list.

Figure 12: Prompt for extracting the Initial Memory Draft.

You are an expert linguist and data parser.
Current Memory Entry: {memory_entry}
Your task: Decompose the complex memory entry above into a list of independent, fine-grained "atomic facts".
Rules:
- Each atomic fact MUST contain only ONE single piece of information (e.g., one action, one time, or one location).
- Each atomic fact must be a complete sentence with a clear subject ("{user_name}").
- Do not add any external information or guess meanings. Just split the existing sentence.
Example:
Entry: "Sarah has a meeting on Sep. 25 at the Oracle office."
Output:
1. Sarah has a meeting.
2. The meeting is on Sep. 25.
3. The meeting is at the Oracle office.
Output format: Just a list of atomic facts, one per line. No other text.

Figure 13: Prompt for Atomic Decomposition (P_{decomp}).

You are a strict factual adjudicator.
An extracted atomic fact has been flagged as potentially hallucinated or inaccurate by an NLI system.
Source Dialogue Turn: {source_turn}
Flagged Atomic Fact: {atomic_fact}
Your task: Read the source dialogue turn carefully and evaluate the flagged fact. Choose one of two actions:
1. **REVISE:** If the fact contains a minor error (e.g., wrong date or slight entity mismatch) but the core intent exists in the dialogue, revise it to perfectly match the dialogue.
2. **DISCARD:** If the fact is a pure hallucination or unsupported inference not mentioned in the dialogue, discard it entirely.
Output format: Output strictly as a JSON object: {{"action": "REVISE" or "DISCARD", "revised_fact": "your revised text or null"}}

Figure 14: Prompt for LLM Intervention and Correction.

You are an analytical investigator building a memory graph.
We have identified two memory events and a potential logical relation between them.
Event A: {event_A}
Event B: {event_B}
Inferred Relation: {relation_type} (from Event A to Event B)
Your task: Generate a specific probing question to verify if this relation truly exists in the user's history.
Rules:
- The question must focus on the LINK between the two events.
- Example: If Event A is "User likes Python" and Event B is "User learns PyTorch", with relation "Causal", the question should be: "Why did the user choose to learn PyTorch?" or "Is the user learning PyTorch because they like Python?"
Output format: Just output the generated question. No other text.

Figure 15: Prompt for generating Relation Probe Questions (P_{probe}).

G AI Assistance Statement

We used Gemini3 for polishing the writing to enhance clarity and readability.